Loan Default Risk Predictor

Project Narrative & Analysis Report
Economic Logic • STAR Framework • Challenges & Solutions • Key Findings
Data Source: Kaggle — Loan Default Dataset (255,347 records, 18 features)
Method: Binary Classification (Logistic Regression, Random Forest)
Deployment: Streamlit Community Cloud

1. Why This Project Exists — and Why It Matters

Every loan a lender issues carries a risk: the borrower may not repay. Lenders have always priced and screened for this risk, but the process has historically leaned on rigid rules and individual underwriter judgment. Machine learning offers a way to estimate that risk systematically from historical patterns — not to replace judgment, but to make it more consistent and data-grounded.

This project builds that estimation from first principles on a real, publicly available dataset, with an explicit focus on doing the hard part honestly: the outcome being predicted (default) is rare — only 11.6% of borrowers in this dataset defaulted. A large share of introductory classification projects quietly ignore this and report a misleadingly high accuracy number. This project treats the imbalance as the central problem to be solved, not a detail to skip past.

Who Needs This — and Why

- Lenders and fintech credit teams screening loan applications for risk before disbursal.
- Risk/underwriting analysts who want a data-driven second opinion alongside traditional credit scoring.
- Students and early-career data scientists learning to handle imbalanced classification honestly — precision/recall trade-offs, not just accuracy.

2. Economic Logic Behind the Project

Credit risk prediction sits at the center of how lending markets function. A lender's core economic problem is asymmetric information: the borrower knows their own repayment intent and financial pressures far better than the lender does. Statistical default models exist to partially correct that asymmetry by using observable borrower and loan characteristics (income, employment history, credit score, debt-to-income ratio) as a proxy for the risk the lender cannot observe directly.

The cost structure of this problem is asymmetric, which is the key economic fact this project treats as a design decision, not an afterthought. A missed defaulter (a 'false negative') costs the lender the full unpaid loan principal and interest. A false alarm (a 'false positive' — flagging a borrower who would actually have repaid) costs comparatively little: typically just the friction of an extra manual review or a slightly more conservative offer. Because these costs are not symmetric, a model tuned purely to maximize accuracy is economically the wrong target — it will happily under-flag the rare, expensive class to look good on paper. This project deliberately re-weights training toward recall on the default class for exactly this reason.

The feature-importance results also carry economic meaning, not just statistical interest: interest rate and income emerging as strong predictors is consistent with standard credit-risk theory (higher rates are correlated with riskier borrower pools; lower income reduces repayment capacity relative to obligations). Age emerging as the single strongest predictor is a more interesting, less textbook-standard finding — discussed

with appropriate caution in Section 5.

3. STAR Framework — Situation, Task, Action, Result

S — SITUATION	Loan default is a rare event (11.6% of records) relative to the majority outcome (repayment). A naive classifier that always predicts 'no default' would already score ~88% accuracy while providing zero practical value to a lender — it would never flag a single risky borrower. No single evaluation metric captures what actually matters here; accuracy alone is actively misleading.
T — TASK	Build, evaluate, and deploy a binary classifier that predicts loan default probability from borrower and loan features, using evaluation metrics appropriate to an imbalanced problem (precision, recall, ROC-AUC), and ship it as a usable, interactive tool — not just a notebook with a final accuracy number.
A — ACTION	Five stages were carried out: (1) exploratory analysis to quantify the class imbalance before any modeling; (2) categorical encoding and a stratified train/test split to preserve the true default rate in both sets; (3) training two models — Logistic Regression as an interpretable baseline and Random Forest for non-linear pattern capture — both using class-weighting rather than discarding data via undersampling; (4) evaluating both on precision, recall, ROC-AUC, and a confusion matrix rather than accuracy alone, and extracting feature importances; (5) selecting the lighter, near-equally-performing Logistic Regression model for deployment and building a live Streamlit interface around it.
R — RESULT	A working classifier achieving ROC-AUC 0.753 (Logistic Regression) and 0.754 (Random Forest) — near-identical ranking ability between a simple and a complex model, which itself is a finding worth reporting rather than defaulting to the more complex option. The deployed model catches approximately 70% of actual defaulters (recall), deliberately trading off precision (~22%) to do so. A live, interactive Streamlit application was built and deployed, allowing real-time default-probability estimation from user-entered borrower details.

4. Challenges Faced and How They Were Tackled

Challenge 1: Class imbalance masking model uselessness

An 11.6% default rate means accuracy is a misleading headline metric — a model predicting 'no default' every time scores ~88% while being useless. Tackled by evaluating on precision, recall, and ROC-AUC from the start, and treating a high accuracy number with explicit suspicion rather than reporting it as the primary result.

Challenge 2: Choosing between undersampling and class-weighting

Balancing the classes by discarding ~196,000 majority-class rows (undersampling) was a real option, but would throw away genuine information about non-defaulters. Tackled by using class_weight='balanced' instead, which re-weights the loss function during training rather than deleting data — preserving the full dataset while still forcing the model to attend to the minority class.

Challenge 3: Selecting a deployment model under a size/performance trade-off

The best-performing model (Random Forest, 200 trees) produced a 28MB serialized file — deployable, but heavier and slower to load than necessary given it scored only marginally higher (ROC-AUC 0.754 vs. 0.753) than Logistic Regression. Tackled by explicitly comparing the trade-off rather than defaulting to the higher-scoring model, and deploying the 1KB Logistic Regression model for a faster, lighter live app.

Challenge 4: Encoding consistency between training and live prediction

One-hot encoding a single user-submitted row (from the Streamlit form) does not automatically produce the same columns as encoding the full training set — categories absent from a single input row would otherwise cause a column mismatch and a runtime crash. Tackled by saving the exact training column list and reindexing every live input against it, filling any missing dummy columns with zero before prediction.

Challenge 5: Interpreting a non-obvious top predictor (Age)

Age emerged as the single strongest predictor of default — stronger than credit score or income, which is not the standard textbook expectation. Tackled by explicitly flagging this as a correlation the model found in the data, not a validated causal claim, since the model has no way to distinguish a genuine age-risk relationship from an unmeasured confound (e.g., loan-purpose mix varying by age group).

Challenge 6: A requested auto-calculated field turned out to be unsupported by the data

The natural next step for the deployed app was auto-calculating Debt-to-Income Ratio from income and loan terms, rather than asking users to enter it manually. Testing this assumption directly (checking DTI's correlation against every other numeric field) showed essentially zero relationship ($|r| < 0.003$) with any of them — the field was generated independently in this dataset, not computed realistically. Tackled by keeping DTI as a manual input rather than silently computing a value the model was never trained to expect, and surfacing the finding directly in the app's UI rather than hiding the limitation.

Challenge 7: Explaining individual predictions in plain language

A raw probability number (e.g. '77% default risk') gives a user no way to understand or act on the result. Tackled by decomposing each prediction using the Logistic Regression model's own coefficients — multiplying each feature's coefficient by its scaled input value to get that feature's exact contribution to that

specific prediction — and surfacing the top contributing factors in plain-language sentences, with an explicit caveat that these reflect statistical association within the model, not causal explanation.

5. Key Findings — What the Data Actually Shows

01 **A simple model matches a complex one on this problem**
Logistic Regression (ROC-AUC 0.753) and Random Forest (ROC-AUC 0.754) perform almost identically. Added model complexity did not meaningfully improve ranking ability here — a reminder that complexity should be justified by measured gain, not assumed.

02 **The model is deliberately biased toward catching defaulters, not toward looking accurate**
The deployed model achieves ~70% recall on actual defaulters at the cost of precision (~22%) — meaning roughly 1 in 5 flagged applicants would have actually repaid. This is a chosen trade-off, justified by the asymmetric cost of a missed default versus a false alarm, not a limitation of the model.

03 **Age is the strongest single predictor of default in this dataset**
Ahead of interest rate, income, and credit score. This is reported as a data pattern, not a causal finding — disentangling a genuine age-related risk effect from a confound would require additional variables not present in this dataset.

04 **Interest rate, income, and employment length behave as economic theory would predict**
Higher interest rates, lower income, and shorter employment history are all associated with higher default probability — consistent with standard credit-risk reasoning, and a useful sanity check that the model learned real signal rather than noise.

6. Limitations — What This Project Cannot Tell You

■ Correlation, not causation

Every relationship reported (Age, Interest Rate, Income, etc.) is a statistical association learned from historical data, not a validated causal mechanism. None of these findings should inform real lending policy without further causal analysis.

■ Debt-to-Income Ratio is not derivable from other fields in this dataset

A natural design choice would be auto-calculating DTI from income and loan repayment terms, as it would be in a real underwriting system. Testing this directly showed DTI has essentially zero correlation ($|r| < 0.003$) with Income, Loan Amount, Interest Rate, Loan Term, Credit Score, or Months Employed in this dataset — meaning it was generated independently rather than computed realistically. Auto-calculating it from other fields would silently feed the model a value unrelated to what it was trained on, so it remains a manual input in the deployed app, with this limitation surfaced directly to the user rather than hidden.

■ Public, likely partially synthetic data

This dataset is a public Kaggle release, originally associated with a Coursera modeling challenge, and may not fully reflect the statistical properties of a real-world loan book. Results should not be generalized to production lending decisions.

■ No temporal dimension

The dataset has no time component — it cannot capture how default risk changes with macroeconomic conditions (interest rate cycles, employment shocks), unlike a true credit-risk model used in production, which typically incorporates a time-varying macro overlay.

■ Precision remains low by design

At ~22% precision, roughly 4 in 5 applicants flagged as high-risk would not actually have defaulted. This is an intentional trade-off favoring recall, but it is a real cost — in a production setting, it would mean a high manual-review burden, which has its own operating cost not modeled here.

■ No fairness/bias audit performed

A production credit model would require an explicit fairness audit across protected characteristics (e.g., checking whether the model's errors are disproportionately concentrated in any demographic group). This project does not perform that audit and should not be treated as fairness-validated.

■ Deployed model is a demonstration, not a production system

The live Streamlit app is built for portfolio and learning purposes. It has no authentication, no audit logging, no monitoring for model drift, and no regulatory compliance layer — all required for any real lending use.